

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_b9d6fa25-e41\agent-  
a87b57c562d87d679.jsonl`  
- SHA-256: `2b5362277fa710a21166e104fff3a9869c1cafa07a8bd283ce4ed98ed06d01b7`  
- Source modified: `2026-06-25T17:36:59+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_b9d6fa25-e41`  
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T17:35:10.741Z)

PR #385 "fix(B1/P9): remove fabricated data from motion segmenter" is CHECKED OUT (branch b1-p9/remove-fabricated-motion-data). The motion segmenter (the LIGHT-tier DEFAULT segmenter) used to FABRICATE rich metadata with the stdlib random module ? server/receiver (random.sample), shot_count (random.randint), avg_speed_kmh (random.uniform), and serve/smash/foul/winner EVENTS ? persisted as if real. This PR removes the fabrication: passes None for the un-modeled fields, DROPS all db.add_event calls, removes the random import. It KEEPS the real signal (segment start/end/duration from pixel-diff detection; intensity derived from detected duration). Changed files (read the real diff: git diff origin/master...HEAD):
backend/pipeline/segmenters/motion.py, tests/test_motion_segmenter.py. CI is GREEN ? do NOT re-run CI; review the CODE.

*** STRICT READ-ONLY: this is a REVIEW. You MUST NOT call Edit/Write/NotebookEdit or modify ANY file. Only Read/Grep/Glob/Bash-to-inspect. If you change a file the review is INVALID. ***

Review ONLY completeness + conventions. (a) ALL db.add_event calls removed? random import + player_pool default block removed? player_pool still on the process_video signature for ABC-compatible + documented as unused? (b) Matches the yolo_hybrid/gemini convention (they already pass server=None/receiver=None)? (c) Net reduction (39/107), NO formatting churn, single-purpose? (d) Is the audit tracker docs/CODE_CLEANUP_AUDIT_2026-06-24.md B1/P9 flipped to ? in THIS PR per the tracked-doc convention? (recall #380's review caught a missing tracker edit ? check git diff origin/master...HEAD for the doc). Report findings + fix.

2. assistant (2026-06-25T17:35:15.203Z)

I'll review PR #385. Let me start by reading the actual diff and the relevant files.

3. user (2026-06-25T17:35:16.381Z)

```
b1-p9/remove-fabricated-motion-data  
eabf774 fix(B1/P9): remove fabricated data from motion segmenter  
cd5b3d2 feat(01/P11): Gemini remote-file GC ? startup orphan sweep (#380)  
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)
```

4. user (2026-06-25T17:35:17.063Z)

```
backend/pipeline/segmenters/motion.py | 100 ++++++-----  
tests/test_motion_segmenter.py        | 46 +++++-----  
2 files changed, 39 insertions(+), 107 deletions(-)
```

5. assistant (2026-06-25T17:35:21.643Z)

Net reduction confirmed (39 insertions / 107 deletions). Let me read the full diff and the changed files.

6. user (2026-06-25T17:35:22.634Z)

```
diff --git a/backend/pipeline/segmenters/motion.py b/backend/pipeline/segmenters/motion.py
index 9b714c5..36d6b36 100644
--- a/backend/pipeline/segmenters/motion.py
+++ b/backend/pipeline/segmenters/motion.py
@@ -1,5 +1,4 @@
 import logging
-import random
 import time
 from typing import Any, Dict, List, Optional, Tuple

@@ -22,12 +21,11 @@ class MotionPlaySegmenter(BasePlaySegmenter):

    def process_video(self, video_id: str, video_path: str, player_pool: Optional[List[str]] =
None, max_frames: Optional[int] = None) -> "Result[List[Dict[str, Any]]]":
        """
-        Processes a video to detect play segments using motion heuristics,
-        populates SQLite with detected segments, and indexes detailed events.
+        Processes a video to detect play segments using motion heuristics and
+        populates SQLite with detected segments. The ``player_pool`` arg is
+        accepted for ABC signature compatibility but no longer used (the motion
+        segmenter has no ground truth for server/receiver ? see _populate_db).
        """
-        if not player_pool:
-            player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
-
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            logger.error(f"Segmenter could not open video: {video_path}")
@@ -35,7 +33,7 @@ class MotionPlaySegmenter(BasePlaySegmenter):

        final_segments = self._detect_motion_segments(cap, max_frames)

-        segments_data = self._populate_db_with_events(video_id, final_segments, player_pool)
+        segments_data = self._populate_db(video_id, final_segments)

        return Success(value=segments_data)

@@ -184,81 +182,35 @@ class MotionPlaySegmenter(BasePlaySegmenter):

        return final_segments

-    def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
-                                player_pool: List[str]) -> List[Dict[str, Any]]:
-        """Persist ``final_segments`` and their fabricated metadata/events to the DB.
+    def _populate_db(self, video_id: str, final_segments: List[Tuple[float, float]]
+                    ) -> List[Dict[str, Any]]:
+        """Persist ``final_segments`` to the DB with honest metadata.

-        For each detected segment this assigns a random server/receiver, fabricates
-        rich tags, then indexes a serve / 0?3 smash / termination event set ? all
-        with the stdlib ``random`` module (the legacy demo's fabricated baseline).
-        Returns the per-segment dicts handed back to the caller. The ``random``
-        call order here is load-bearing for reproducibility and must not change.
+        The motion segmenter detects play segments via pixel-diff heuristics but
+        has NO ground truth for who served, shot counts, shuttle speed, or event
+        breakdown ? those were previously fabricated with the stdlib ``random``
+        module (the legacy demo baseline). They are now persisted as ``None`` /
+        unknown markers so consumers can distinguish "not modeled" from a real
+        value. Matches yolo_hybrid/gemini, which pass ``server=None, receiver=None``.
        """
-        # Populate SQLite DB with segments and metadata events
-        segments_data = []
```

```

for start, end in final_segments:
-     # Assign Server and Receiver randomly from pool
-     server, receiver = random.sample(player_pool, 2)
-
-     # Formulate dynamic rich tags
    dur = end - start
    metadata = {
-         "shot_count": random.randint(4, 25),
+         "shot_count": None,
        "intensity": "high" if dur > 12 else "medium",
-         "avg_speed_kmh": round(random.uniform(40, 85), 1)
+         "avg_speed_kmh": None,
    }

-     segment_id = self.db.add_play_segment(video_id, start, end, server, receiver,
metadata)
-
-     if segment_id:
-         # Add sub-segment events
-         # A: Serve Event (always at start)
-         serve_foul = random.random() < 0.15 # 15% chance of serve foul
-         self.db.add_event(
-             segment_id=segment_id,
-             timestamp=start + 0.5,
-             event_type="serve",
-             player=server,
-             details={"status": "foul" if serve_foul else "valid", "type": "low_serve"
if random.random() > 0.5 else "high_serve"}
-         )
-
-         # B: Smash Events (1 to 3 random smashes per segment)
-         smashes_count = random.randint(0, 3)
-         for _ in range(smashes_count):
-             smash_time = random.uniform(start + 1.5, end - 1.0)
-             smasher = random.choice([server, receiver])
-             self.db.add_event(
-                 segment_id=segment_id,
-                 timestamp=round(smash_time, 2),
-                 event_type="smash",
-                 player=smasher,
-                 details={"speed_kmh": round(random.uniform(120, 250), 1)}
-             )
-
-         # C: Segment Termination Event (foul/winner)
-         foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
-         is_foul = random.random() > 0.4
-         error_player = random.choice([server, receiver])
-
-         self.db.add_event(
-             segment_id=segment_id,
-             timestamp=end - 0.2,
-             event_type="foul" if is_foul else "winner",
-             player=error_player,
-             details={"type": random.choice(foul_types)} if is_foul else {"shot_type":
"drop" if random.random() > 0.5 else "smash"}
-         )
-
-         # Store segment representation to return to caller
-         segments_data.append({
-             "id": segment_id,
-             "start_time": start,
-             "end_time": end,
-             "duration": dur,
-             "server": server,
-             "receiver": receiver,

```

```

-         "metadata": metadata
-     })
-
+     segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)
+     segments_data.append({
+         "id": segment_id,
+         "start_time": start,
+         "end_time": end,
+         "duration": dur,
+         "server": None,
+         "receiver": None,
+         "metadata": metadata,
+     })
+     return segments_data

# Register the segmenter

```

7. user (2026-06-25T17:35:22.864Z)

```

diff --git a/tests/test_motion_segmenter.py b/tests/test_motion_segmenter.py
index 8d6a710..228531e 100644
--- a/tests/test_motion_segmenter.py
+++ b/tests/test_motion_segmenter.py
@@ -18,7 +18,6 @@ extracted into private helpers without changing observable output:
     fabricated server/receiver/shot_count/events are reproducible. The random call
     ORDER inside process_video is what the seeded assertions pin.
     """
-import random
-from unittest.mock import MagicMock, patch

import backend.pipeline.segmenters.motion as motion_mod
@@ -145,42 +144,36 @@ def _high_motion_ratios():
     return [0.5] * 40

-def test_process_video_fabricates_one_segment_deterministically():
-    """With seeded ``random`` the fabricated segment + metadata are pinned exactly."""
+def test_process_video_emits_one_segment_with_honest_metadata():
+    """B1/P9: the motion segmenter persists honest None/unknown metadata (no fabricated
+    server/receiver/shot_count/speed/events). Boundaries still come from the motion loop."""
     cv2_mock, _ = _make_cv2_mock(_high_motion_ratios())

     db_mock = MagicMock()
-    # add_play_segment returns a deterministic id so the returned dict is pinned.
     db_mock.add_play_segment.return_value = 42

     segmenter = _make_segmenter(db_mock=db_mock)

     with patch.object(motion_mod, "cv2", cv2_mock):
-        random.seed(0)
         result = segmenter.process_video("vid-1", "active.mp4")

     assert isinstance(result, Success)
     segments = result.value

-    # === PIN: exactly one fabricated segment survives the min-duration filter ===
+    # === PIN: exactly one segment survives the min-duration filter ===
     assert len(segments) == 1
     seg = segments[0]

-    # Pin the full returned dict shape/values (boundaries come from the motion
-    # loop, server/receiver/metadata from seeded random).
+    # Boundaries from the motion loop; metadata is honest (None / unknown).
     assert seg == _EXPECTED_SEGMENT

```

```

- # === PIN: the DB side effects ===
+ # === PIN: the DB side effects ? server/receiver are None, no fabrication ===
db_mock.add_play_segment.assert_called_once()
ps_args, ps_kwargs = db_mock.add_play_segment.call_args
assert _EXPECTED_ADD_PLAY_SEGMENT_ARGS == (ps_args, ps_kwargs)

- # Events fabricated for the segment (serve + N smashes + termination).
- assert db_mock.add_event.call_count == _EXPECTED_EVENT_COUNT
- actual_events = [
-     (call.args, call.kwargs) for call in db_mock.add_event.call_args_list
- ]
- assert actual_events == _EXPECTED_EVENTS
+ # B1/P9: NO events are fabricated anymore (was serve+smashes+termination).
+ db_mock.add_event.assert_not_called()

# Captured from a characterization run against the UNCHANGED motion.py with
@@ -195,26 +188,13 @@ _EXPECTED_SEGMENT = {
    "start_time": _START,
    "end_time": _END,
    "duration": 7.999999999999999,
-   "server": "Deepak",
-   "receiver": "Suresh",
-   "metadata": {"shot_count": 5, "intensity": "medium", "avg_speed_kmh": 51.7},
+   "server": None,          # B1/P9: no ground truth ? was fabricated "Deepak"
+   "receiver": None,       # B1/P9: no ground truth ? was fabricated "Suresh"
+   "metadata": {"shot_count": None, "intensity": "medium", "avg_speed_kmh": None},
}

_EXPECTED_ADD_PLAY_SEGMENT_ARGS = (
-   ("vid-1", _START, _END, "Deepak", "Suresh",
-    {"shot_count": 5, "intensity": "medium", "avg_speed_kmh": 51.7}),
+   ("vid-1", _START, _END, None, None,
+    {"shot_count": None, "intensity": "medium", "avg_speed_kmh": None}),
    {},
)
-
- _EXPECTED_EVENT_COUNT = 4
-
- _EXPECTED_EVENTS = [
-     ((), {"segment_id": 42, "timestamp": 0.6666666666666666, "event_type": "serve",
-          "player": "Deepak", "details": {"status": "valid", "type": "high_serve"}}),
-     ((), {"segment_id": 42, "timestamp": 6.99, "event_type": "smash",
-          "player": "Suresh", "details": {"speed_kmh": 195.8}}),
-     ((), {"segment_id": 42, "timestamp": 6.66, "event_type": "smash",
-          "player": "Deepak", "details": {"speed_kmh": 156.6}}),
-     ((), {"segment_id": 42, "timestamp": 7.966666666666666, "event_type": "foul",
-          "player": "Suresh", "details": {"type": "net_touch"}}),
- ]

```

8. assistant (2026-06-25T17:35:25.943Z)

The core diff looks clean. Now let me verify the claims systematically. First, check the full current motion.py to confirm no leftover `db.add\_event` or `random` usage, and that `player\_pool` remains on the signature.

9. user (2026-06-25T17:35:26.349Z)

```

1     import logging
2     import time
3     from typing import Any, Dict, List, Optional, Tuple
4
5     import cv2
6     import numpy as np
7

```

```

8     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
9     from backend.results import Failure, Result, Success # standardized contract
10
11     logger = logging.getLogger(__name__)
12
13     class MotionPlaySegmenter(BasePlaySegmenter):
14         @property
15         def name(self) -> str:
16             return "motion"
17
18         @property
19         def description(self) -> str:
20             return "Traditional CV pixel-motion heuristics tracking frame differences."
21
22         def process_video(self, video_id: str, video_path: str, player_pool:
Optional[List[str]] = None, max_frames: Optional[int] = None) -> "Result[List[Dict[str, Any]]]":
23             """
24             Processes a video to detect play segments using motion heuristics and
25             populates SQLite with detected segments. The ``player_pool`` arg is
26             accepted for ABC signature compatibility but no longer used (the motion
27             segmenter has no ground truth for server/receiver ? see _populate_db).
28             """
29             cap = cv2.VideoCapture(video_path)
30             if not cap.isOpened():
31                 logger.error(f"Segmenter could not open video: {video_path}")
32                 return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
33
34             final_segments = self._detect_motion_segments(cap, max_frames)
35
36             segments_data = self._populate_db(video_id, final_segments)
37
38             return Success(value=segments_data)
39
40         def _detect_motion_segments(self, cap: Any, max_frames: Optional[int] = None) ->
List[Tuple[float, float]]:
41             """Run the pixel-motion heuristic over ``cap`` and return the detected
42             (start, end) play segments.
43
44             Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
45             thresholds it into raw segments, then merges dead-time gaps and filters
46             out segments shorter than ``min_segment_duration``. ``cap`` is released
47             before returning. This is the detection half of ``process_video`` ? it
48             performs no DB writes and no fabrication.
49
50             ``max_frames`` limits the number of analysed frames (Optional; for debugging).
51             An explicit ``np.ones((3,3))`` kernel is used for ``cv2.dilate`` rather than
52             ``None`` so the typed overload resolves correctly.
53             """
54             fps = cap.get(cv2.CAP_PROP_FPS)
55             total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
56             duration = total_frames / fps if fps > 0 else 0
57
58             # Sub-sample settings (5 FPS for analysis)
59             analysis_fps = 5.0
60             frame_interval = max(1, int(fps / analysis_fps))
61
62             idx_cfg = self.config.indexing
63             motion_threshold = idx_cfg.motion_threshold
64             min_segment_duration = idx_cfg.min_segment_duration
65             dead_time_merge_gap = idx_cfg.dead_time_merge_gap
66
67             prev_gray = None
68             motion_signals = []
69             timestamps = []

```

```

70
71     frame_idx = 0
72     processed_count = 0
73     t_start = time.time()
74     # Emit a progress line roughly every 5% of the video so long runs are
75     # observable instead of silent (a full 1080p match is ~tens of minutes).
76     progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
77     next_progress = progress_step
78     logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
79               f"(~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
80     while True:
81         # Skip decoding for intermediate frames using cap.grab() for high
performance
82         for _ in range(frame_interval - 1):
83             if not cap.grab():
84                 break
85             frame_idx += 1
86
87         ret, frame = cap.read()
88         if not ret:
89             break
90
91         # Process frame
92         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
93         # Apply Gaussian Blur to smooth noise
94         gray = cv2.GaussianBlur(gray, (21, 21), 0)
95
96         t = frame_idx / fps
97         timestamps.append(t)
98
99         if prev_gray is not None:
100             # Frame absolute difference (highly optimized motion tracking)
101             diff = cv2.absdiff(prev_gray, gray)
102             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
103
104             # Dilate to fill gaps
105             kernel = np.ones((3, 3), np.uint8)
106             thresh = cv2.dilate(thresh, kernel, iterations=2)
107
108             # Motion ratio
109             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
110             motion_signals.append(motion_ratio)
111         else:
112             motion_signals.append(0.0)
113
114         prev_gray = gray
115         frame_idx += 1
116         processed_count += 1
117
118         if progress_step and frame_idx >= next_progress:
119             pct = 100.0 * frame_idx / total_frames
120             elapsed = time.time() - t_start
121             eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
122             logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
123                       f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
124             next_progress += progress_step
125
126         if max_frames is not None and processed_count >= max_frames:
127             logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
128             break

```

```

129
130         if frame_idx >= total_frames:
131             break
132
133     cap.release()
134
135     # Smooth signal using a moving average
136     window_size = 5
137     smoothed = []
138     for i in range(len(motion_signals)):
139         start = max(0, i - window_size // 2)
140         end = min(len(motion_signals), i + window_size // 2 + 1)
141         smoothed.append(sum(motion_signals[start:end]) / (end - start))
142
143     # Detect active play boundaries
144     in_segment = False
145     segment_start: Optional[float] = None
146     raw_segments: List[Tuple[float, float]] = []
147
148     for i, t in enumerate(timestamps):
149         signal = smoothed[i]
150         if signal > motion_threshold:
151             if not in_segment:
152                 in_segment = True
153                 segment_start = t
154             else:
155                 if in_segment:
156                     in_segment = False
157                     assert segment_start is not None # set when in_segment became True
158                     raw_segments.append((segment_start, t))
159
160     # Handle video ending while in active segment
161     if in_segment:
162         assert segment_start is not None # set when in_segment became True
163         raw_segments.append((segment_start, timestamps[-1]))
164
165     # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
166     merged_segments: List[Tuple[float, float]] = []
167     if raw_segments:
168         curr_start, curr_end = raw_segments[0]
169         for next_start, next_end in raw_segments[1:]:
170             if next_start - curr_end < dead_time_merge_gap:
171                 # Merge
172                 curr_end = next_end
173             else:
174                 merged_segments.append((curr_start, curr_end))
175                 curr_start, curr_end = next_start, next_end
176         merged_segments.append((curr_start, curr_end))
177
178     # Post-Processing: 2. Filter segments shorter than min_segment_duration
179     final_segments: List[Tuple[float, float]] = [
180         r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
181     ]
182
183     return final_segments
184
185 def _populate_db(self, video_id: str, final_segments: List[Tuple[float, float]])
186     ) -> List[Dict[str, Any]]:
187     """Persist ``final_segments`` to the DB with honest metadata.
188
189     The motion segmenter detects play segments via pixel-diff heuristics but
190     has NO ground truth for who served, shot counts, shuttle speed, or event
191     breakdown ? those were previously fabricated with the stdlib ``random``
192     module (the legacy demo baseline). They are now persisted as ``None`` /
193     unknown markers so consumers can distinguish "not modeled" from a real

```



```

194         value. Matches yolo_hybrid/gemini, which pass ``server=None, receiver=None``.
195         """
196         segments_data = []
197         for start, end in final_segments:
198             dur = end - start
199             metadata = {
200                 "shot_count": None,
201                 "intensity": "high" if dur > 12 else "medium",
202                 "avg_speed_kmh": None,
203             }
204             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
205             segments_data.append({
206                 "id": segment_id,
207                 "start_time": start,
208                 "end_time": end,
209                 "duration": dur,
210                 "server": None,
211                 "receiver": None,
212                 "metadata": metadata,
213             })
214         return segments_data
215
216     # Register the segmenter
217     segmenter_registry.register(MotionPlaySegmenter)
218

```

10. assistant (2026-06-25T17:35:32.896Z)

motion.py is clean: no `random` import, no `db.add\_event`, `player\_pool` retained on the signature for ABC-compat and documented as unused. Now let me verify the convention claim (yolo_hybrid/gemini pass server=None/receiver=None) and check for the audit tracker doc edit.

11. user (2026-06-25T17:35:34.481Z)

```

backend/pipeline/segmenters/gemini.py:507:             segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend/pipeline/segmenters/trajectory_hybrid.py:464:             segment_id =
self.db.add_play_segment(
backend/pipeline/segmenters/yolo_hybrid.py:351:             segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
Binary file backend/pipeline/segmenters/__pycache__/gemini.cpython-313.pyc matches
Binary file backend/pipeline/segmenters/__pycache__/motion.cpython-313.pyc matches
Binary file backend/pipeline/segmenters/__pycache__/trajectory_hybrid.cpython-313.pyc matches
Binary file backend/pipeline/segmenters/__pycache__/yolo_hybrid.cpython-313.pyc matches

```

12. user (2026-06-25T17:35:35.311Z)

```

backend/infrastructure/database.py:498:     def add_event(self, segment_id: int, timestamp:
float, event_type: str, player: Optional[str] = None, details: Optional[Dict[str, Any]] = None)
-> bool:
Binary file backend/pipeline/segmenters/__pycache__/motion.cpython-313.pyc matches

```

13. assistant (2026-06-25T17:35:38.144Z)

Convention confirmed: gemini.py:507 and yolo_hybrid.py:351 both pass `server=None, receiver=None` exactly as motion.py now does. Let me check the trajectory_hybrid variant and confirm `add\_event` is no longer used by any segmenter (only defined in the DB layer).

14. user (2026-06-25T17:35:39.248Z)

```

        "shot_count_confidence", "Unknown"
    ),
    "detector": f"{self.family}-hybrid-{model_name}",

```

```

}
segment_id = self.db.add_play_segment(
    video_id, start, end, None, None, metadata
)
if segment_id:
    candidate_id = segment.get("_candidate_id")
    if candidate_id is not None:
        self.db.link_candidate_to_segment(candidate_id, segment_id)

```

15. user (2026-06-25T17:35:40.446Z)

```

tests/test_database.py:43: db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
tests/test_database.py:467: def test_add_event_write_failure_returns_false_and_logs(tmp_path,
monkeypatch, caplog):
tests/test_database.py:471: ok = db.add_event(1, 5.0, "smash")

```

16. assistant (2026-06-25T17:35:45.002Z)

Confirmed: after this PR, no segmenter calls `add\_event`. The remaining `add\_event` references are the DB-layer definition and its own DB tests (`test\_database.py`), which legitimately exercise the DB method directly ? not affected by this PR and correct to leave alone. trajectory_hybrid also passes `None, None` ? fully consistent with the convention.

Now the key check from the prompt: is the audit tracker `docs/CODE\_CLEANUP\_AUDIT\_2026-06-24.md` B1/P9 flipped to ? in THIS PR? Let me check whether the doc exists, what it says about B1/P9, and whether it's in the diff.

17. user (2026-06-25T17:35:46.130Z)

```

backend/pipeline/segmenters/motion.py
tests/test_motion_segmenter.py

```

18. user (2026-06-25T17:35:47.263Z)

```

docs/CODE_CLEANUP_AUDIT_2026-06-24.md
---EXISTS---

```

19. assistant (2026-06-25T17:35:50.136Z)

The audit doc exists but is NOT in the PR diff (only motion.py and test file changed). This is the exact concern the prompt flagged (recalling #380's missing tracker edit). Let me read the doc's B1/P9 entry to confirm its current state and whether it needs flipping.

20. user (2026-06-25T17:35:51.521Z)

```

56:| **B1** | **HIGH** | `backend/pipeline/segmenters/motion.py` | **Fabricated data.** Random
values for server/receiver/shot_count/events/intensity/avg_speed_kmh persisted as real data. | 2
|
99:| **Q2** | `backend/pipeline/segmenters/motion.py` | Fix return type annotation: `List[Dict]`
? `Result[List[Dict[str, Any]]]`. | 1 |
104:| **Q7** | `mypy.ini` | Type-clean `motion.py` (small, legacy) as a ratchet exercise. | 2 |
133:- `backend/pipeline/segmenters/motion.py` ? uses `self.config.get("indexing", {})`
166:- **Does NOT change any behavior.** The Gemini provider still returns `[]` on failure; the
motion segmenter still fabricates events; the PTS validator still has a hardcoded 10.0s gap.
187:| **T1** | **Tier 1: Q2 (motion return type) + D11 (codec docstring)** | P0 | No | ? |
[#362](https://github.com/Khelsutra/badminton-highlight-indexer/pull/362) |
197:| **P9** | **B1: Remove fabricated data from motion segmenter** | P7 | No | ? | ? |
203:| **P15** | **Q7: Type-clean `motion.py`** ? ratchet exercise | P7 | No | ? |
[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |
238:| ~1~ | ~**B2 / P17**~ | ~**Finish** the `Success`/`Failure` return contract~ ? **?
DONE (this PR).** The 3 callers that dropped provider error metrics now surface
`metrics["error"]` loudly. **Follow-up remains:** migrate the hybrid segmenters' `process_video`
return type from bare `List[Dict]` to the ABC's `Result[...]` (`Success`/`Failure`) ? currently
suppressed with `type: ignore[override]` in `trajectory_hybrid.py` (motion.py is already fully

```

migrated). |

245:| 3 | **B1 / P9** | Remove fabricated data from `motion.py` | Random values for `server`/`receiver`/`shot\_count`/`events`/`avg\_speed\_kmh` are persisted as real data. Replace with `None` or honest "unknown" markers. |

255:> **Removed from this list ? already shipped** (see the \$6 tracker); they were stale entries left from before Round 3/4 landed: **D4/P13** deep config merge (#373 ? `loader.py::\_deep\_merge`), **02/P14** single-executor risk documented (#374 ? `job\_worker.py`), **Q7/P15** `motion.py` type-cleaned (#373 ? no longer in the `mypy.ini` quarantine list).

280:- `2026-06-25` ? **P17** done (**B2** finish). **The 4 callers that dropped provider error metrics** (`trajectory\_hybrid.py`, `yolo\_hybrid.py`, `gemini\_refine.refine\_window`, `gemini\_refine.full\_video\_rallies`) now check `metrics["error"]` and log loudly instead of silently treating a provider failure as "0 rallies" ? matching the pattern the `gemini` segmenter already uses. (`yolo\_hybrid` was caught during PR #378 review ? it bound `metrics` but never read it.) Suite 1484/1484 green; ruff + mypy clean. Follow-up: migrate the hybrids' `process\_video` return type from bare `List[Dict]` to the ABC `Result[...]` contract (motion.py is already migrated; hybrids suppressed with `type: ignore[override]` for now).

21. assistant (2026-06-25T17:35:56.604Z)

This confirms the finding. The audit doc has the B1/P9 tracker row at line 197 still showing `?` (unchecked), and it is NOT touched in this PR's diff. Per the project's tracked-doc convention (and the exact precedent flagged from #380's review), the tracker should be flipped to ? with the PR link in the same PR that completes the work. Let me read the relevant doc sections to give a precise fix.

22. user (2026-06-25T17:35:57.885Z)

183 | **P1** | **D3-a: Convert validators to typed `AppConfig`** (6 files) | P0 | No | ? |

[#355](https://github.com/Khelsutra/badminton-highlight-indexer/pull/355) |

184 | **P6a** | **Q3: Deduplicate `\_default\_db\_path`** | P0 | No | ? |

[#356](https://github.com/Khelsutra/badminton-highlight-indexer/pull/356) |

185 | **P6b** | **Q4: Cache builtin defaults** | P0 | No | ? |

[#358](https://github.com/Khelsutra/badminton-highlight-indexer/pull/358) |

186 | **P6c** | **Q5: requirements.txt torch comment + Q6: isort (I) + Q8: cv2 comment** |

P0 | No | ? |

[#359](https://github.com/Khelsutra/badminton-highlight-indexer/pull/359)

[#360](https://github.com/Khelsutra/badminton-highlight-indexer/pull/360)

[#361](https://github.com/Khelsutra/badminton-highlight-indexer/pull/361) |

187 | **T1** | **Tier 1: Q2 (motion return type) + D11 (codec docstring)** | P0 | No | ? |

[#362](https://github.com/Khelsutra/badminton-highlight-indexer/pull/362) |

188 | **T2** | **Tier 2: B6 (resolution_fps fallback + warning log)** | P0 | No | ? |

[#363](https://github.com/Khelsutra/badminton-highlight-indexer/pull/363) |

189 | **T3** | **Tier 3+4: D12 (sys.path hack) + B3 (PTS gap config) + O5 (lock note) + D7 (centralize TUNED)** | P0 | No | ? |

[#364](https://github.com/Khelsutra/badminton-highlight-indexer/pull/364) |

190 | **P6d** | **~D7: Centralize hardcoded tuning constants~ ? folded into T3** | P0 | No | ? |

[#364](https://github.com/Khelsutra/badminton-highlight-indexer/pull/364) |

191 | **P2** | **D3-b: Convert segmenters to typed `AppConfig`** (15 files: 5 segmenters + 4 detectors + worker + eval + tools) | P1 | No | ? |

[#366](https://github.com/Khelsutra/badminton-highlight-indexer/pull/366) |

192 | **P3** | **D3-c: Remove `.get()` compat shim; flip `extra='forbid'`** (18 files, +87/-169) | P2 | No | ? |

[#367](https://github.com/Khelsutra/badminton-highlight-indexer/pull/367) |

193 | **P5** | **D2: App-factory refactor** ? replace module-level Optional globals with `create\_app(config, db) ? FastAPI` | P3, P4 | No | ? |

[#370](https://github.com/Khelsutra/badminton-highlight-indexer/pull/370) |

194 | **P7** | **Phase-1 verification:** full suite green, coverage ? 84%, mypy ratchet |

P1?P6 | No | ? |

1463/1463 pass (#371); `backend.api.job\_worker` lifted from mypy quarantine |

195 | ? | ? | ? | ? | ? | ? | ? |

196 | **P8** | **B2: Error-flagging in Gemini provider metrics** (Phase 2) | P7 | No | ? |

[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |

197 | **P9** | **B1: Remove fabricated data from motion segmenter** | P7 | No | ? | ? |

198 | **P10** | **D1: Split `main.py` god-module** (~1450 lines ? smaller modules) | P7 | No | ? | ? |

199 | **P11** | **O1: Gemini remote-file GC** ? per-call `\_safe\_delete` (#373) + startup orphan sweep scoped to our `UPLOAD\_TAG`-tagged files, grace-gated, HTTP-timeout-bounded | P7 |

```

No | ? | [#380](https://github.com/Khelsutra/badminton-highlight-indexer/pull/380) |
200 | **P12** | **D13: ByteTrack deprecation** ? migrate to standalone `trackers` package |
P7 | No | ? | ? |
201 | **P13** | **D4: Deep config merge** ? section-level replace ? recursive merge | P7 |
No | ? | [#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |
202 | **P14** | **O2: Single-executor risk** ? documented with mitigations in job_worker.py
| P7 | No | ? | [#374](https://github.com/Khelsutra/badminton-highlight-indexer/pull/374) |
203 | **P15** | **Q7: Type-clean `motion.py`** ? ratchet exercise | P7 | No | ? |
[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |
204 | **P17** | **B2 (cont.): finish the `Success`/`Failure` return contract** ? the 4
callers that dropped provider error metrics (`trajectory_hybrid`, `yolo_hybrid`,
`gemini_refine.refine_window`, `gemini_refine.full_video_rallies`) now surface
`metrics["error"]` loudly instead of silently treating a failure as "0 rallies". (Full
`Success`/`Failure` return-type migration of the hybrids is follow-up.) | P8 | No | ? |
[#378](https://github.com/Khelsutra/badminton-highlight-indexer/pull/378) |
205 | **P16** | Phase 3 items (B4/B7/B8/D5/D6/D8-D10/D14/O3/O4) | P15 | No | ? | ? |
206
207 ---
208
209 ## 7. Open questions
210
211 1. **Q: Should D1 (split `main.py`) be Phase 1 or Phase 2** ? Leaning Phase 2: D2 (app-
factory) is a natural precursor, and D1 is lower risk after the globals are gone. Owner to
confirm.
212 2. ~~Q: Should Phase 1 include enabling `I` (isort) in ruff.toml?~~ ? ? Shipped in
#360.
213
214 ---
215
216 ## 8. Definition of done
217
218 - [x] P0: This doc approved and merged to `master`
219 - [x] P1?P3: Typed-config migration complete ? no `.get()` dict access remains in
backend code; `extra='forbid'` on `AppConfig`
220 - [x] P4: `segmenter_must_be_registered` actually validates against the registry
221 - [x] P5: `create_app()` injects shared state via `app.state` (handlers read
`request.app.state`); `backend.api.job_worker` removed from mypy quarantine. (The Optional
module globals `db_instance`/`global_config` are **retained as a transitional back-compat shim**
and `backend.main` stays mypy-quarantined ? full removal is follow-up, see D1/P10.)
222 - [x] P6: Quick wins Q1?Q6, Q8 landed
223 - [x] P7: Full test suite ? 1463/1463 pass (#371); `ruff` + `mypy` clean;
`backend.api.job_worker` lifted from quarantine
224 - [x] All PRs are single-purpose, branched from `master`, with descriptive commit
messages
225 - [ ] `docs/README.md` updated to reflect the project's terminal state
226 - [ ] Final status flipped to `DONE` and this doc moved to
`docs/archives/past_projects/`
227
228 ---
229
230 ## 8.1 ? Next items ? Phase 2 pick-up (prioritized)
231
232 All items below are **Claude-doable**, small-PR-friendly, and unblocked. Pick up in
order:
233
234 ### Immediate (HIGH impact, low risk)
235
236 | # | ID | What | Why |
237 |---|---|-----|-----|
238 | ~1~ | ~**B2 / P17**~ | ~**Finish** the `Success`/`Failure` return contract~ ?
**? DONE (this PR).** The 3 callers that dropped provider error metrics now surface
`metrics["error"]` loudly. **Follow-up remains:** migrate the hybrid segmenters' `process_video`
return type from bare `List[Dict]` to the ABC's `Result[...]` (`Success`/`Failure`) ? currently
suppressed with `type: ignore[override]` in `trajectory_hybrid.py` (motion.py is already fully
migrated). |

```

```

239      | ~-2~- | ~**01 / P11**~- | ~-Gemini remote-file GC~- ? **? DONE (#380).** Per-call
`_safe_delete` (#373) + startup orphan sweep scoped to `UPLOAD_TAG`-tagged files, grace-gated,
HTTP-timeout-bounded. |
240
241      ### Near-term (MEDIUM impact)
242
243      | # | ID | What | Why |
244      |---|----|-----|-----|

```

23. assistant (2026-06-25T17:36:03.838Z)

This fully confirms the finding. The §6 tracker row P9 (line 197) is still `?` with no PR link, and the §8.1 "Next items" list at line 245 still lists B1/P9 as an open pick-up item. Neither is updated in this PR's diff. The precedent (P8/#373, P11/#380, P17/#378) is that the completing PR flips its own tracker row to ? with the PR link.

Let me confirm the changelog section and check there's nothing in the diff touching the doc at all, plus verify the §6 changelog convention (line 255+ has dated entries that this PR would normally add to).

24. user (2026-06-25T17:36:05.076Z)

```

244      |---|----|-----|-----|
245      | 3 | **B1 / P9** | Remove fabricated data from `motion.py` | Random values for
`server`/`receiver`/`shot_count`/`events`/`avg_speed_kmh` are persisted as real data. Replace
with `None` or honest `"unknown"` markers. |
246      | 4 | **D13 / P12** | ByteTrack deprecation migration | `sv.ByteTrack` removed in
supervision 0.30.0; pinned `<0.30.0` as stopgap. Migrate to standalone `trackers` package. |
247      | 5 | **D1 / P10** | Split `main.py` god-module | ~1450 lines. D2 (app-factory) was the
necessary precursor ? now done. Extract: CLI handlers ? `backend/cli/`, upload routes ? already
in `backend/api/uploads.py`, debug helpers ? `scripts/`. |
248
249      ### Later (deferred)
250
251      | # | ID | What |
252      |---|----|-----|
253      | 6+ | **P16+** | B4, B7, B8, D5, D6, D8?D10, D14, O3, O4 ? Phase 3 items |
254
255      > **Removed from this list ? already shipped (see the §6 tracker); they were stale
entries left from before Round 3/4 landed.** D4/P13 deep config merge (#373 ?
`loader.py::_deep_merge`), O2/P14 single-executor risk documented (#374 ? `job_worker.py`),
Q7/P15 `motion.py` type-cleaned (#373 ? no longer in the `mypy.ini` quarantine list).
256
257      ---
258
259      ## 9. References
260
261      - **Audit source:** 33 findings from a deep whole-repo read on 2026-06-24, plus 2
absorbed from `docs/BACKLOG.md` (D13, D14) for a total of 35. Two additional BACKLOG items (D2,
O1) were already independently found and are cross-referenced in §3e.
262      - **Prior hardening sweep:** `docs/CODE_AUDIT_AND_TEST_HARDENING.md` (completed
2026-06-14) ? the previous audit-driven hardening round. This project is the next wave.
263      - **BACKLOG.md** `docs/BACKLOG.md` ? deferred items from the 2026-06-10 sweep; §3e maps
the absorbed items.
264      - **CODE_MAP.md** `docs/CODE_MAP.md` ? placement rules, gotchas, and the cross-cutting
footguns list.
265      - **mypy.ini** ? quarantine list and the ratchet instruction.
266      - **ruff.toml** ? lint rules; `"I"` (isort) deferred.
267      - **PROJECT_DOC_TEMPLATE.md** `docs/PROJECT_DOC_TEMPLATE.md` ? the template this doc
follows.
268
269      ---
270
271      ### Changelog
272      - `2026-06-24` ? Initial doc created from 33-item deep audit.

```

273 - `2026-06-24` ? Absorbed 2 findings from `docs/BACKLOG.md` (D13, D14). Cross-referenced D2 and 01 in §3e.

274 - `2026-06-24` ? Phase 1 begun. P0 (#352) ? #353 ? P4/B5 (#354) ? P1/D3-a (#355) ? P6a/Q3 (#356). Status ? IN PROGRESS.

275 - `2026-06-24` ? **Round 2.** Quick wins (#358 Q4, #359 Q5, #360 Q6 isort, #361 Q8) ? Tier 1 (#362 Q2+D11) ? Tier 2 (#363 B6) ? Tier 3+4 (#364 D12+B3+05). 11 items shipped total.

276 - `2026-06-24` ? **Round 3.** P2/D3-b segmenters (#366) + P3/D3-c extra='forbid' (#367). 18 items shipped.

277 - `2026-06-24` ? **Round 4 (final).** P5/D2 app-factory (#370) + test fixes (#371). Status ? PHASE 1 COMPLETE. Suite 1463/1463 green. §8.1 added with prioritized Phase 2 pick-up.

278 - `2026-06-25` ? **§8.1 reconciled with the §6 source-of-truth (docs-only).** Removed three already-shipped items that §8.1 still listed as deferred future work ? D4/P13, 02/P14, Q7/P15 (all merged in #373/#374) ? and split the unfinished half of B2 out of the done P8 into a new **P17** (the provider return contract + the `trajectory\_hybrid`/`gemini\_refine` callers that still drop `metrics["error"]`). Grounded against `loader.py::\_deep\_merge`, `mypy.ini`, `backend/results.py`, and `backend/providers/gemini.py` + its callers. No code change.

279 - `2026-06-25` ? **Accuracy fixes (docs-only).** (1) §8 DoD P5 said the module globals were "replaced with `app.state`"; corrected ? `create\_app()` *added* `app.state` and lifted `job\_worker` from quarantine, but `db\_instance`/`global\_config` remain as a transitional shim and `backend.main` is still mypy-quarantined (`backend/main.py:264`, `mypy.ini`). (2) `main.py` size "~1300" ? "~1450" (now 1448 lines) in §3b/§6/§8.1.

280 - `2026-06-25` ? **P17 done (B2 finish).** The 4 callers that dropped provider error metrics (`trajectory\_hybrid.py`, `yolo\_hybrid.py`, `gemini\_refine.refine\_window`, `gemini\_refine.full\_video\_rallies`) now check `metrics["error"]` and log loudly instead of silently treating a provider failure as "0 rallies" ? matching the pattern the `gemini` segmenter already uses. (`yolo\_hybrid` was caught during PR #378 review ? it bound `metrics` but never read it.) Suite 1484/1484 green; ruff + mypy clean. Follow-up: migrate the hybrids' `process\_video` return type from bare `List[Dict]` to the ABC `Result[...]` contract (motion.py is already migrated; hybrids suppressed with `type: ignore[override]` for now).

281

25. assistant (2026-06-25T17:36:11.219Z)

The convention is crystal-clear and consistently followed: every completed item flips its §6 tracker row to ? with the PR link, removes itself from the §8.1 pick-up list, and adds a dated §changelog entry ? and this is done in the same PR that lands the work (P17/#378, 01/#380 both did exactly this). PR #385 does none of it. This is precisely the gap #380's review caught, recurring here.

Let me also do a final check on the line at 166 ("the motion segmenter still fabricates events") which is now a stale claim, and verify the `\_\_init\_\_.py` / any other consumer doesn't break on `server=None` ? though that's behavioral, the prompt scopes me to completeness + conventions. Let me confirm line 166's context.

26. user (2026-06-25T17:36:12.390Z)

160 **Test strategy:** All existing tests must pass byte-identically. The `TestClient` usage changes from `from backend.main import app` to `from backend.main import create\_app; app = create\_app(test\_config)`.

161

162 ---

163

164 ## 5. Honest limits ? what Phase 1 does NOT do

165

166 - **Does NOT change any behavior.** The Gemini provider still returns `[]` on failure; the motion segmenter still fabricates events; the PTS validator still has a hardcoded 10.0s gap.

167 - **Does NOT fix B2 (Success/Failure in Gemini).** That changes the return contract callers depend on ? deferred to Phase 2.

168 - **Does NOT split `main.py` (D1).** The god-module split is deferred ? D2 (app-factory) is a necessary precursor.

169 - **Does NOT address any operational risks (01?05).** Those need behavioral changes.

170 - **A green suite after Phase 1 does NOT mean the code is "clean."** It means the structural debt targeted by D2/D3/B5 is resolved. The remaining 28 items still need attention.

171

27. assistant (2026-06-25T17:36:20.346Z)

Line 166 is a §5 "Honest limits ? what **Phase 1** does NOT do" statement. It is a historical, Phase-1-scoped claim ("the motion segmenter still fabricates events"), accurate as of Phase 1. Since B1/P9 is a Phase 2 item, this line is contextually frozen-in-time and not strictly a regression ? but it does become globally stale once this PR lands. I'll note it as a NIT under the same doc-update umbrella, not a blocker on its own.

Let me confirm there are no other consumers reading ``seg["server"]`/`metadata["shot_count"]`` assuming non-None (a quick completeness sanity check on whether the None change is conventionally safe ? sibling segmenters already emit None so consumers must already tolerate it, but worth a fast grep).

28. user (2026-06-25T17:36:22.064Z)

```
backend/infrastructure/database.py:606:         params.append(filters["server"])
backend/infrastructure/database.py:610:         params.append(filters["receiver"])
backend/main.py:517:         sc = (row.get("shot_count") or "").strip()
backend/main.py:520:         meta["shot_count"] = int(float(sc)) # OverflowError
on 'inf'/'1e400'
backend/main.py:522:         meta["shot_count"] = sc
backend/main.py:1005:         sc = meta.get("shot_count") if isinstance(meta, dict) else None
backend/tools/export.py:73:         shot = _segment_metadata(s).get("shot_count")
backend/tools/export.py:146:         shot = md.get("shot_count", "")
backend/worker/__main__.py:78:         w.writerow(["start", "end", "shot_count",
"ending_reason"])
backend/worker/__main__.py:83:         s.get("shot_count", ""),
```

29. assistant (2026-06-25T17:36:31.324Z)

These consumers all use `.get()` with defaults or None-tolerant handling (e.g. ``meta.get("shot_count")`, `md.get("shot_count", "")`, `s.get("shot_count", "")``), so passing ``None`` is conventionally safe ? and the sibling segmenters already emit ``None``, so consumers must already tolerate it. The behavioral change is sound.

I have everything I need. Summary of the review:

- ***(a) Completeness ? code***: ? All ``db.add_event`` calls removed; ``random`` import removed; ``player_pool`` default-pool block (``["Avi", "Suresh", "Ramesh", "Deepak"]``) removed. ``player_pool`` retained on ``process_video`` signature for ABC-compat and explicitly documented as unused (lines 22-28). Real signal preserved (start/end/duration + duration-derived intensity).
- ***(b) Convention***: ? Now matches `gemini.py:507` and `yolo_hybrid.py:351` (and `trajectory_hybrid.py:464`) which all pass ``server=None, receiver=None``. No segmenter calls ``add_event`` anymore.
- ***(c) Single-purpose***: ? Net reduction 39/107, only `motion.py` + its test touched, no formatting churn.
- ***(d) Tracker doc***: ? **NOT** flipped. The audit doc is absent from the PR diff. This is the gap flagged from #380's review, recurring.

30. user (2026-06-25T17:36:48.848Z)

Structured output provided successfully

31. assistant (2026-06-25T17:36:58.968Z)

Review complete. PR #385's **code is clean and complete** on every code-level criterion the prompt asked about:

- ***(a)*** All ``db.add_event`` calls removed; ``random`` import removed; ``player_pool`` default-pool block removed. ``player_pool`` is retained on the ``process_video`` signature for ABC-compat and explicitly documented as unused (``backend/pipeline/segmenters/motion.py:22-28`, `:185-195``).
- ***(b)*** Matches the sibling convention exactly ? ``gemini.py:507`, `yolo_hybrid.py:351`, and `trajectory_hybrid.py:464`` all pass ``server=None, receiver=None``, and after this PR no segmenter calls ``add_event`` (only the DB-layer definition + its own DB tests remain).

- **(c)** Single-purpose, net reduction 39/107, only ``motion.py`` + its test touched, no formatting churn. Consumers all read these fields None-tolerantly (``get()`` with defaults), so the ``None`` switch is safe.

The one real gap is **(d)**: the tracked audit doc ``docs/CODE_CLEANUP_AUDIT_2026-06-24.md`` is **absent from the PR diff**, so B1/P9 is still ``?`` at §6 line 197 with no PR link, still listed as an open pick-up at §8.1 line 245, and has no ``$changelog`` entry ? exactly the missing-tracker-edit that #380's review caught, recurring here. I flagged that as MEDIUM with the precise lines to fix, plus a NIT for the now-stale §5 line 166 claim to fold into the same doc edit. Per the strict read-only constraint, I made no file changes.